Project viva preparation

24 questions grounded in the supplied implementation

Use the code references to support each answer. The system is a local simulator. Role availability, normalized energy, and subnet sensor delivery must be explained within their modeled boundaries.

Environment note: Mesa 3.5.1 conflicts with runner.py assigning model.scenario. Verification uses Mesa 3.3.1. Pin a compatible release and include SciPy for the study analyzer.

01. Why separate logical hierarchy from physical mesh?

Membership defines role candidates and aggregation. Physical adjacency determines whether their representatives can actually communicate.

Evidence: simulation/entities.py; simulation/topology.py

02. How many assignments does the default world require?

Four roles across ten subnet, three network, and one internetwork scopes: 4 × 14 = 56.

Evidence: simulation/config.py; ProtocolEngine.assignment_keys

03. Does anomaly_detection execute a detection model?

No. It is a role label scored from energy and health. No anomaly classifier or sensor-payload detector is implemented.

Evidence: nodes.py; simulation/runner.py

04. How are node scores calculated?

ROLE_WEIGHTS defines normalized weighted sums. Gateway uses .40 energy, .35 link quality, and .25 initial degree centrality.

Evidence: nodes.py

05. What happens to a failed node during aggregation?

Its capability becomes zero but its child weight remains in the denominator, reducing the parent aggregate.

Evidence: nodes.py; simulation/capability.py

06. Is the root aggregate the global average node score?

Not necessarily. Default aggregation averages immediate children at each level, and the networks have unequal descendant counts.

Evidence: simulation/capability.py; SimulationConfig.subnet_counts

07. How is a container represented on the physical mesh?

The first available descendant node in its stored order represents it. This is an abstraction rather than optimized representative election.

Evidence: EntityHierarchy.representative_node_id

08. What prevents optional migration oscillation?

Candidate capability minus weighted migration cost must strictly exceed current capability plus hysteresis. Cooldown and adaptive hysteresis are optional.

Evidence: ProtocolEngine._consider_cost_aware_migration

09. How is migration cost different from charged energy?

The score cost combines estimated messages, transfer, route-based energy, and disruption. Actual router counters charge the modeled exchanges that execute.

Evidence: calculate_migration_cost; ProtocolEngine._migrate

10. How do lease renewal and epoch retention interact?

Renewal updates lease timing at its configured fraction. Without retention, epoch changes regrant leases, potentially occurring before long leases renew.

Evidence: ProtocolEngine._renew_if_due; _advance_epoch

11. Does epoch fencing solve distributed split-brain?

No. Epoch labels and invalidation operate in one process. Independent receivers, delayed messages, and stale-epoch rejection are not simulated.

Evidence: simulation/protocols.py; simulation/leases.py

12. Why does routing sometimes choose more hops?

Dijkstra minimizes quality-weighted cost, so two reliable edges can cost less than one poor edge.

Evidence: NetworkRouter._current_graph; _routes_from

13. When are cached routes invalidated?

Failure, recovery, link degradation, and depletion invalidate usable graph and source-route caches.

Evidence: simulation/runner.py; NetworkRouter.invalidate_graph

14. What is the routing complexity caveat?

Heap-based shortest paths are followed by explicit reconstruction of every destination path. Path storage can dominate textbook distance-only bounds.

Evidence: NetworkRouter._routes_from

15. How is solar energy modeled?

A constant seeded per-node rate charges batteries up to capacity. Overflow is curtailed, and depleted nodes rejoin after reaching reserve.

Evidence: RoleSimulationModel._harvest_solar_energy

16. Can the solar gate block failure recovery?

No. The gate restricts optional migrations. Assignment after holder failure takes the recovery path instead.

Evidence: ProtocolEngine.step; _solar_migration_is_worthwhile

17. What does CSC measure?

The fraction of required role-time intervals whose lease and holder satisfy role_available. It excludes time zero and does not prove service-task execution.

Evidence: MetricsRecorder.record; ProtocolEngine.role_available

18. Is sensor delivery measured at the root?

No. It counts successful sensor items reaching subnet gateways. Optional aggregate backhaul has separate counters.

Evidence: RoleSimulationModel._route_sensor_traffic

19. Why can high residual energy coexist with poor service?

A failed controller or unavailable gateway can suppress traffic and leave nodes alive but unable to provide modeled service.

Evidence: simulation/runner.py; stored per_run_metrics.csv

20. What does right-censored lifetime mean?

The threshold did not cross during the observation horizon. The horizon is a bound on observation, not a measured failure time.

Evidence: simulation/metrics.py; patent-study/analyze_patent_study.py

21. Why compare paired seeds?

The arms share topology and event schedules for each seed/scenario, reducing unrelated world variation in within-seed differences.

Evidence: patent-study/run_patent_study.py

22. What does D minus C establish about the gate?

In the stored study it does not establish an overall delivery or energy-efficiency improvement. Normal delivery is unchanged and energy/item slightly rises.

Evidence: patent-study/results_20000/paired_comparison.csv

23. What security mechanisms exist?

Input validation and state consistency checks exist. Authentication, encryption, authorization, and adversarial-node defenses do not.

Evidence: simulation/config.py; simulation/leases.py

24. What should be improved first?

Define real role-task execution and end-to-end item delivery, then add asynchronous communication and calibrated energy before deployment claims.

Evidence: Architectural recommendation based on implementation